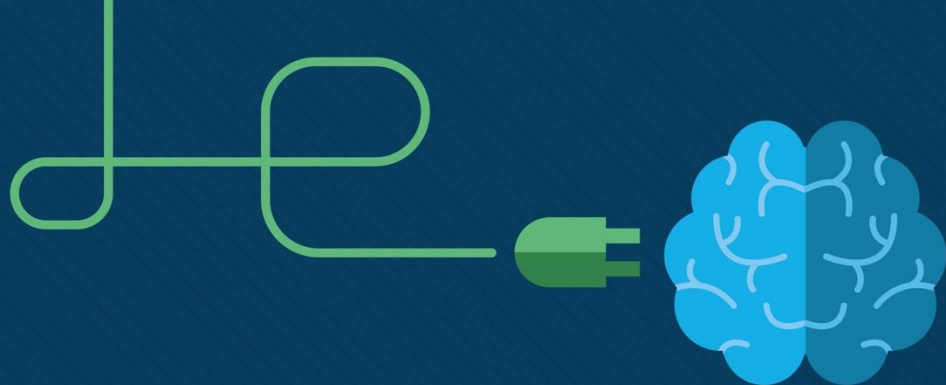




Chapter 9: Transport Layer

CCNA Routing and Switching
Introduction to Networks v6.0



Chapter 9 - Sections & Objectives

■ 9.1 Transport Layer Protocols

- Explain how transport layer protocols and services support communications across data networks.
- Explain the purpose of the transport layer in managing the transportation of data in end-to-end communication.
- Explain characteristics of the TCP and UDP protocols, including port numbers and their uses.

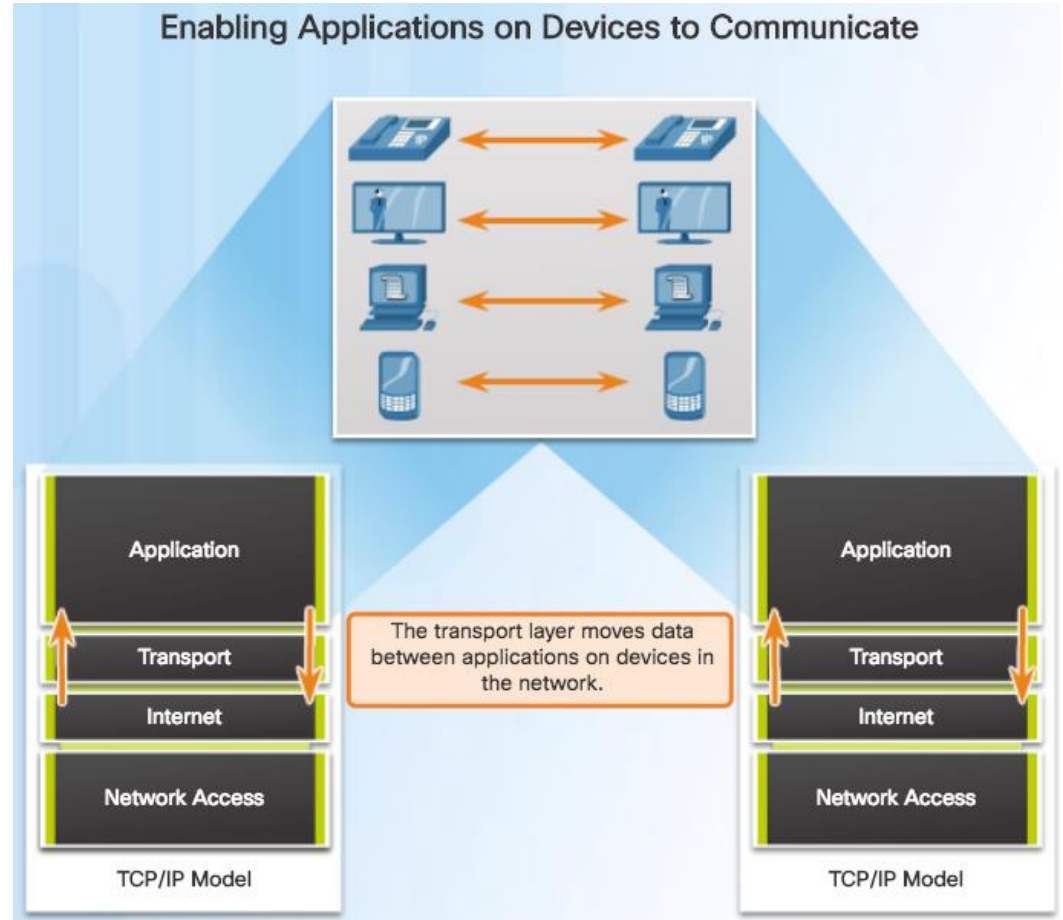
■ 9.2 TCP and UDP

- Compare the operations of transport layer protocols in supporting end-to-end communication.
- Explain how TCP session establishment and termination processes facilitate reliable communication.
- Explain how TCP protocol data units are transmitted and acknowledged to guarantee delivery.
- Describe the UDP client processes to establish communication with a server.
- Determine whether high-reliability TCP transmissions, or non-guaranteed UDP transmissions, are best suited for common applications.

9.1 Transport Layer Protocols

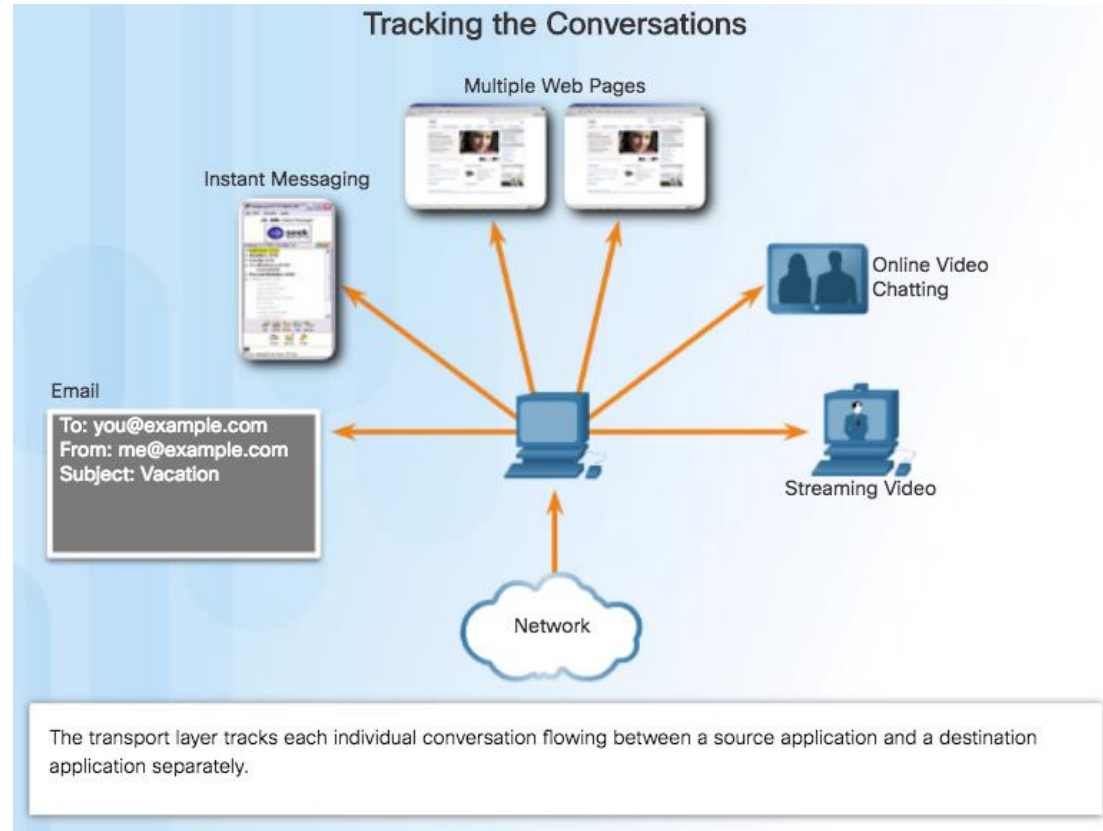
Role of the Transport Layer

- Responsible for establishing a temporary communication session between two applications and delivering data between them.
- Link between the application layer and the lower layers that are responsible for network transmission.



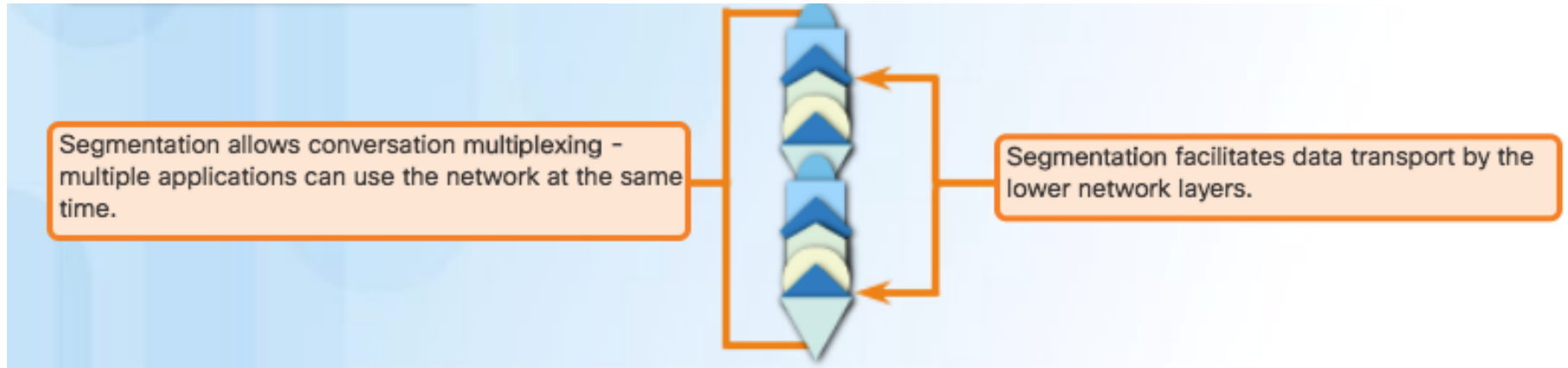
Transport Layer Responsibilities

- **Tracking the Conversation** - Tracks each individual conversation flowing between a source and a destination application.
- **Segmentation** - Divides the data into segments that are easier to manage and transport. Header used for reassembly is used for tracking.
- **Identifying the Application** - Ensures that even with multiple applications running on a device, all applications receive the correct data via port numbers.



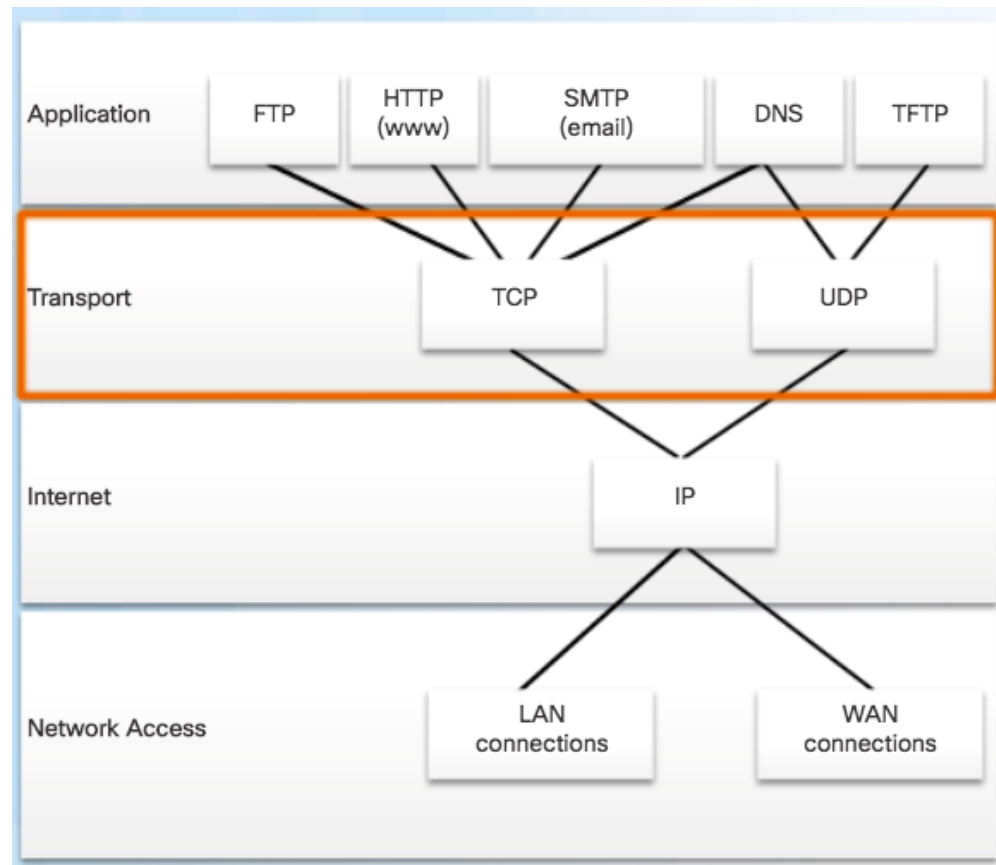
Conversation Multiplexing

- Segmenting the data into smaller chunks enables many different communications to be multiplexed on the same network.



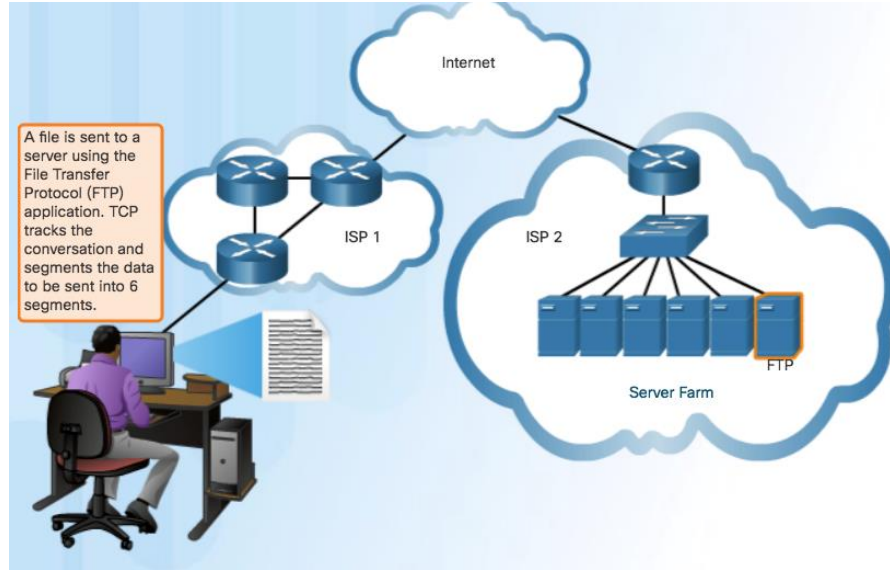
Transport Layer Reliability

- TCP/IP provides two transport layer protocols:
 - Transmission Control Protocol (TCP)
 - Considered reliable which ensures that all of the data arrives at the destination.
 - Additional fields needed in header which increases size and delay.
- User Datagram Protocol (UDP)
 - Does not provide for reliability.
 - Fewer fields and is faster than TCP.

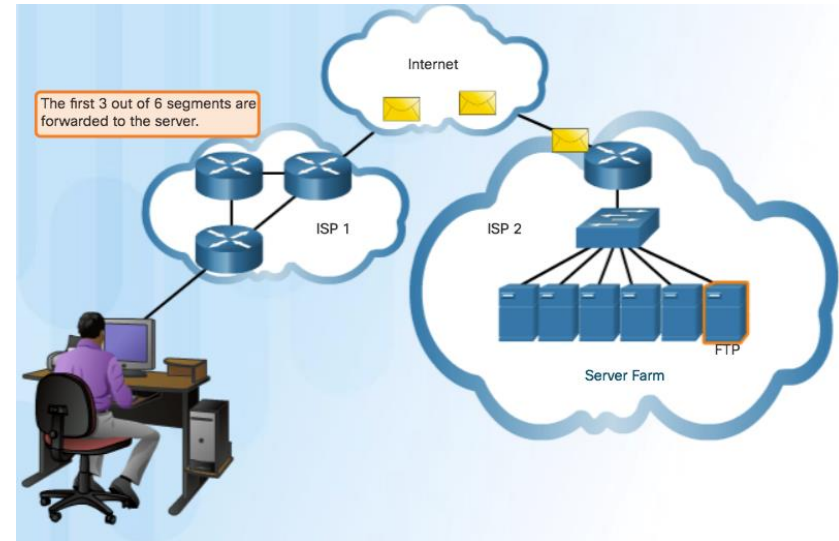


Transportation of Data

TCP

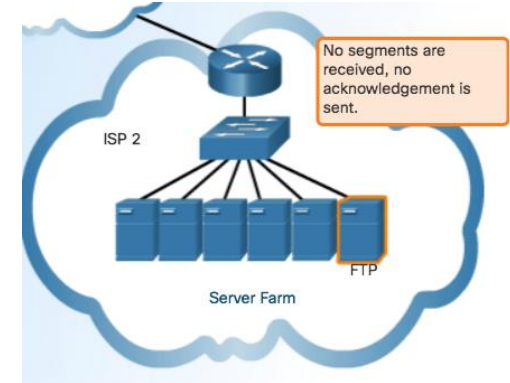
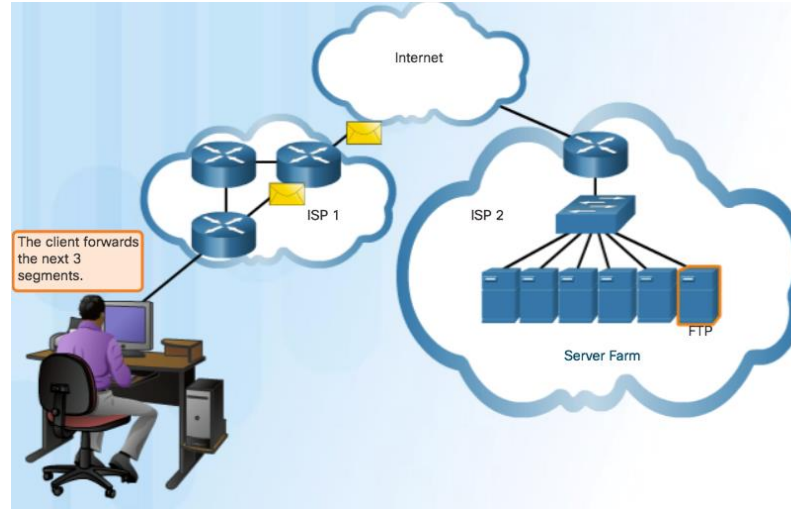
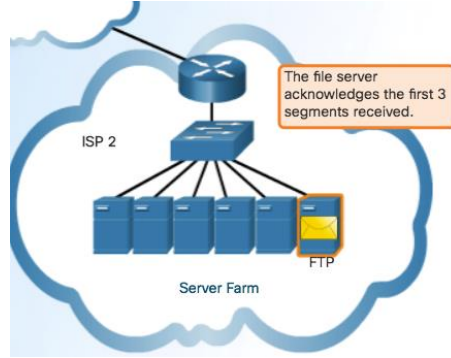


- TCP transport is similar to sending tracked packages. If a shipping order is broken up into several packages, a customer can check online to see the order of the delivery.



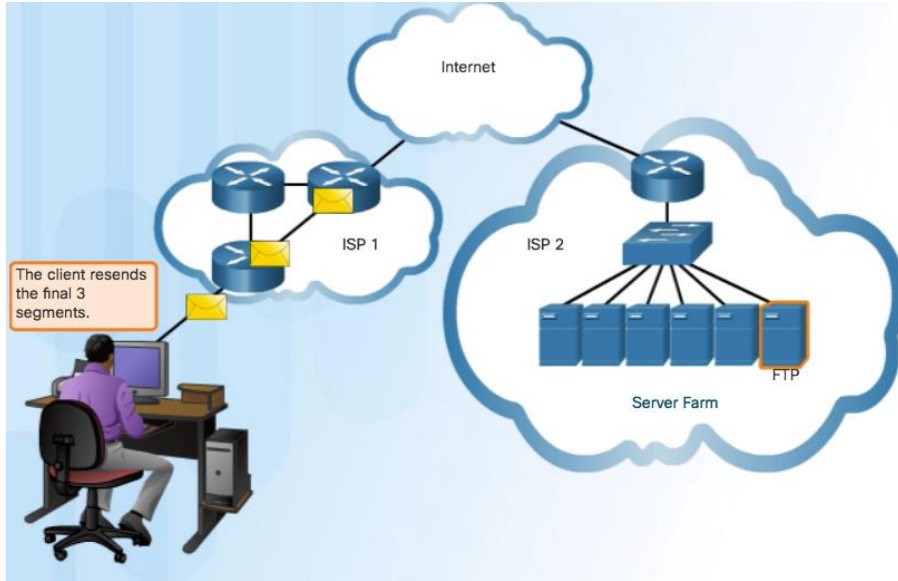
Transportation of Data

TCP (Cont.)



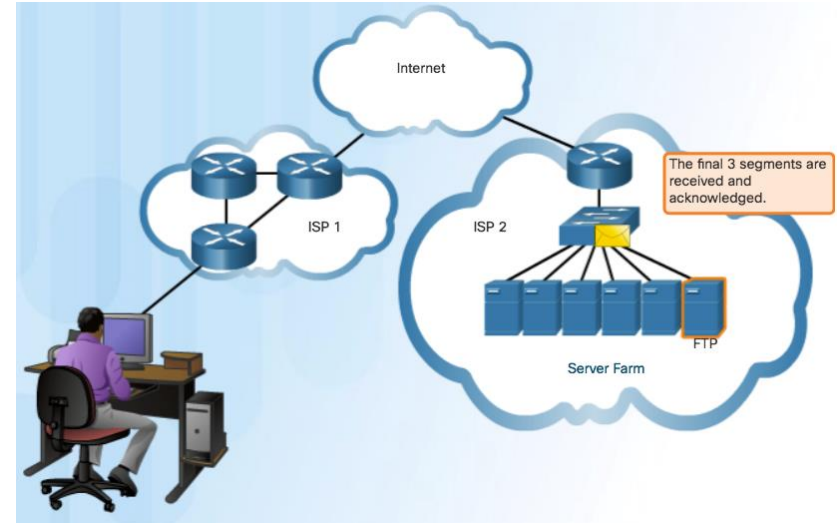
Transportation of Data

TCP (Cont.)



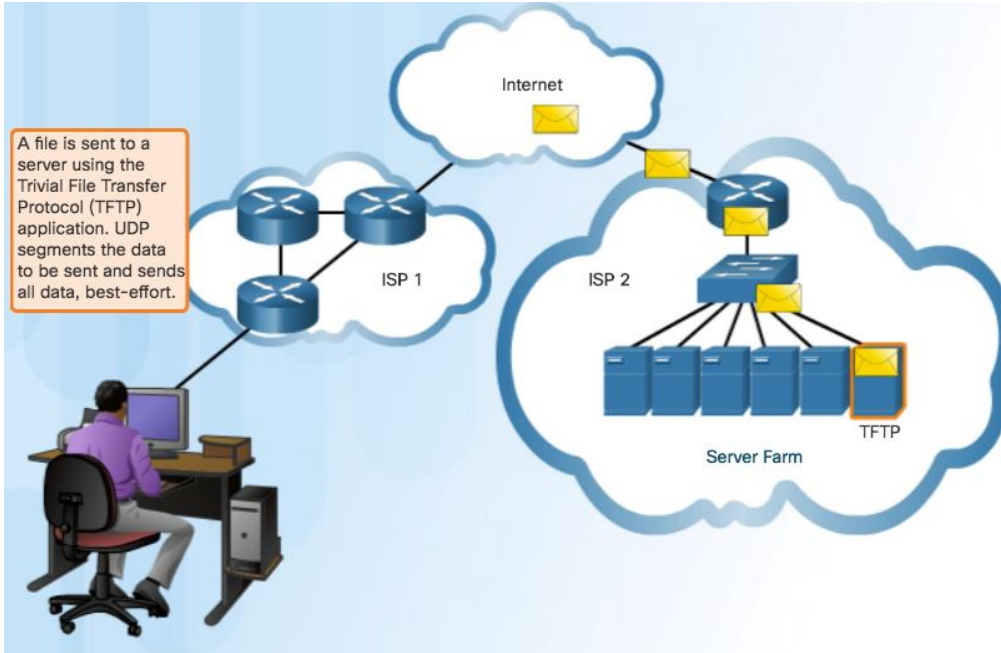
TCP Three Responsibilities:

- Numbering and tracking data segments
- Acknowledging received data
- Retransmitting any unacknowledged data after a certain period of time



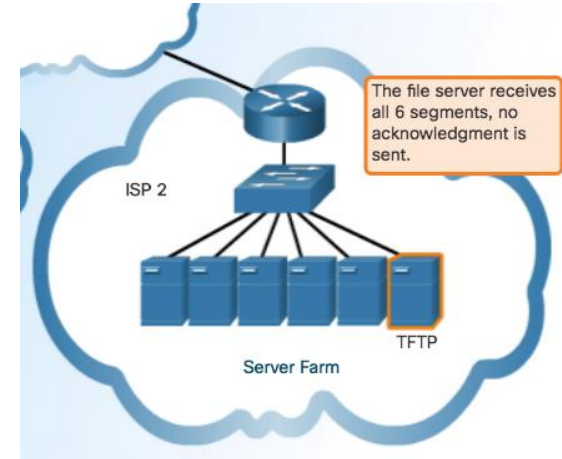
Transportation of Data

UDP



Use UDP for less overhead and to reduce possible delays.

- Best-effort delivery (unreliable)
- No acknowledgment
- Similar to a non-registered letter



The Right Transport Layer Protocol for the Right Application

- TCP - databases, web browsers, and email clients require that all data that is sent arrives at the destination in its original condition.
- UDP - if one or two segments of a live video stream fail to arrive, if disruption in the stream, may not be noticeable to the user.

UDP



IP Telephony



Streaming Live Video

Required protocol properties:

- Fast
- Low overhead
- Does not require acknowledgements
- Does not resend lost data
- Delivers data as it arrives

TCP



SMTP/POP
(Email)



HTTP

Required protocol properties:

- Reliable
- Acknowledges data
- Resends lost data
- Delivers data in sequenced order

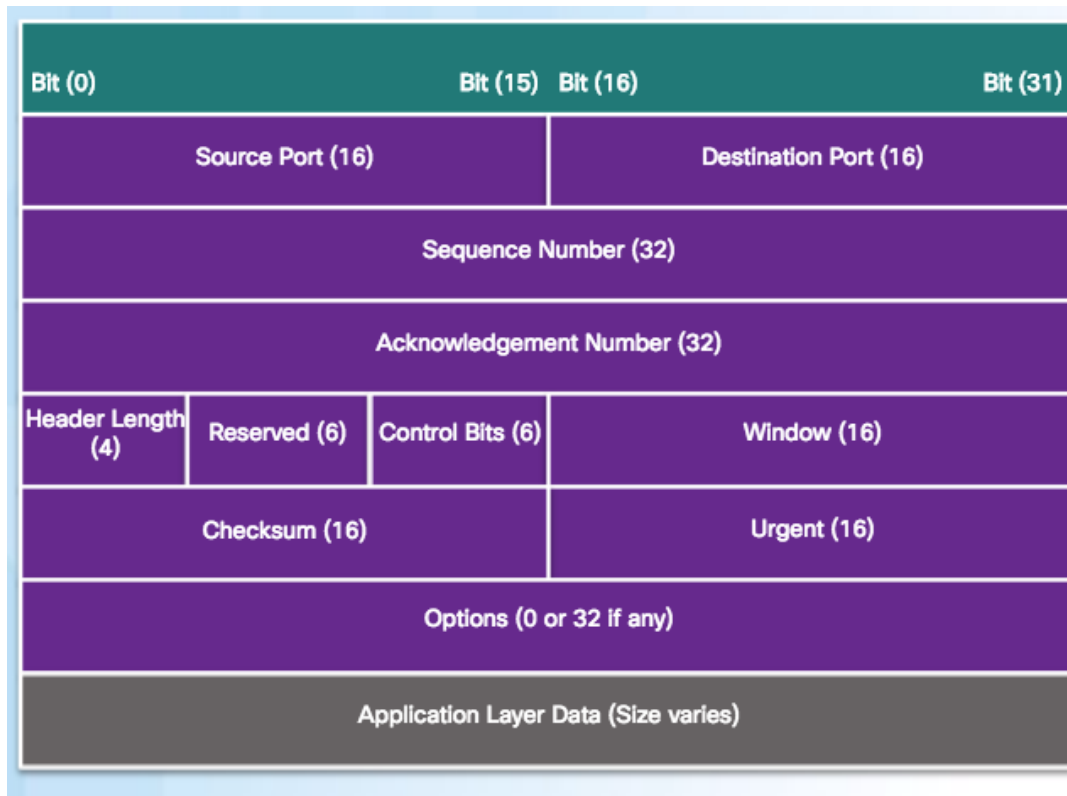
TCP Features

- Establishing a Session
 - Connection-oriented protocol
 - Ensures the application is ready to receive the data
 - Negotiate the amount of traffic that can be forwarded at a given time
- Reliable Delivery
 - Ensuring that each segment that the source sends arrives at the destination
- Same-Order Delivery
 - Numbering & Sequencing the segments guarantees reassembly into the proper order
- Flow Control
 - Regulate the amount of data the source transmits

TCP Header

- Source and Destination Port used to identify application
- Sequence number used for data reassembly
- Acknowledgement number indicates data has been received and ready for next byte from source
- Header length – length of TCP segment header
- Control bits – purpose and function of TCP segment
- Window size – number of bytes that can be accepted at one time
- Checksum – Used for error checking of segment header and data

20 Bytes Total



TCP and UDP Overview

UDP Features



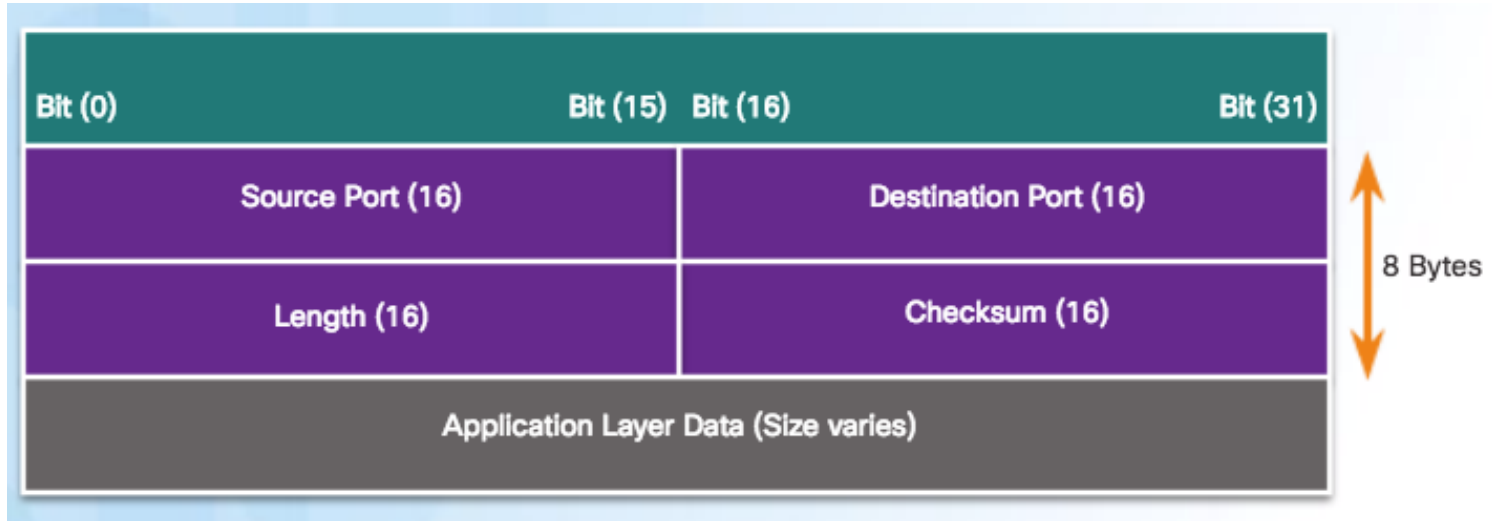
Features of UDP

- Data is reconstructed in the order that it is received.
- Any segments lost are not resent.
- No session establishment.
- Does not inform the sender about resource availability.

TCP and UDP Overview

UDP Header

- UDP is a stateless protocol – no tracking
- Reliability handled by application



Multiple Separate Communications

- Users expect to simultaneously receive and send email, view websites and make a VoIP phone call
- TCP and UDP manage multiple conversations by using unique identifiers called port numbers



TCP and UDP Overview

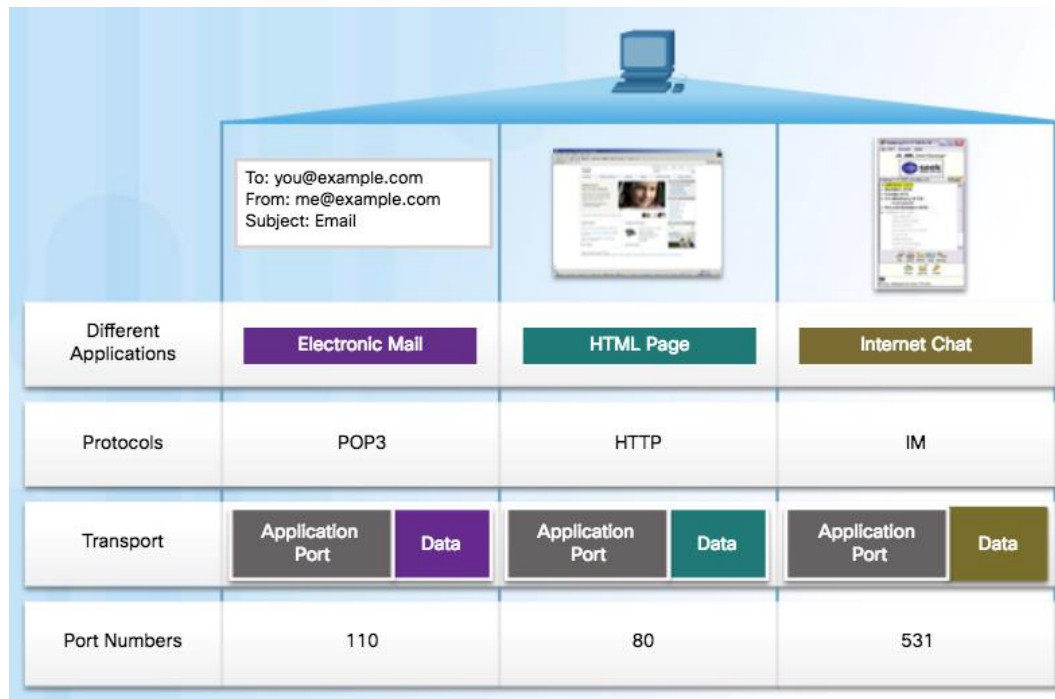
Port Numbers

■ Source Port

- Originating application port that is dynamically generated by sending device
- Example: Each separate HTTP conversation is tracked based on the source ports.

■ Destination Port

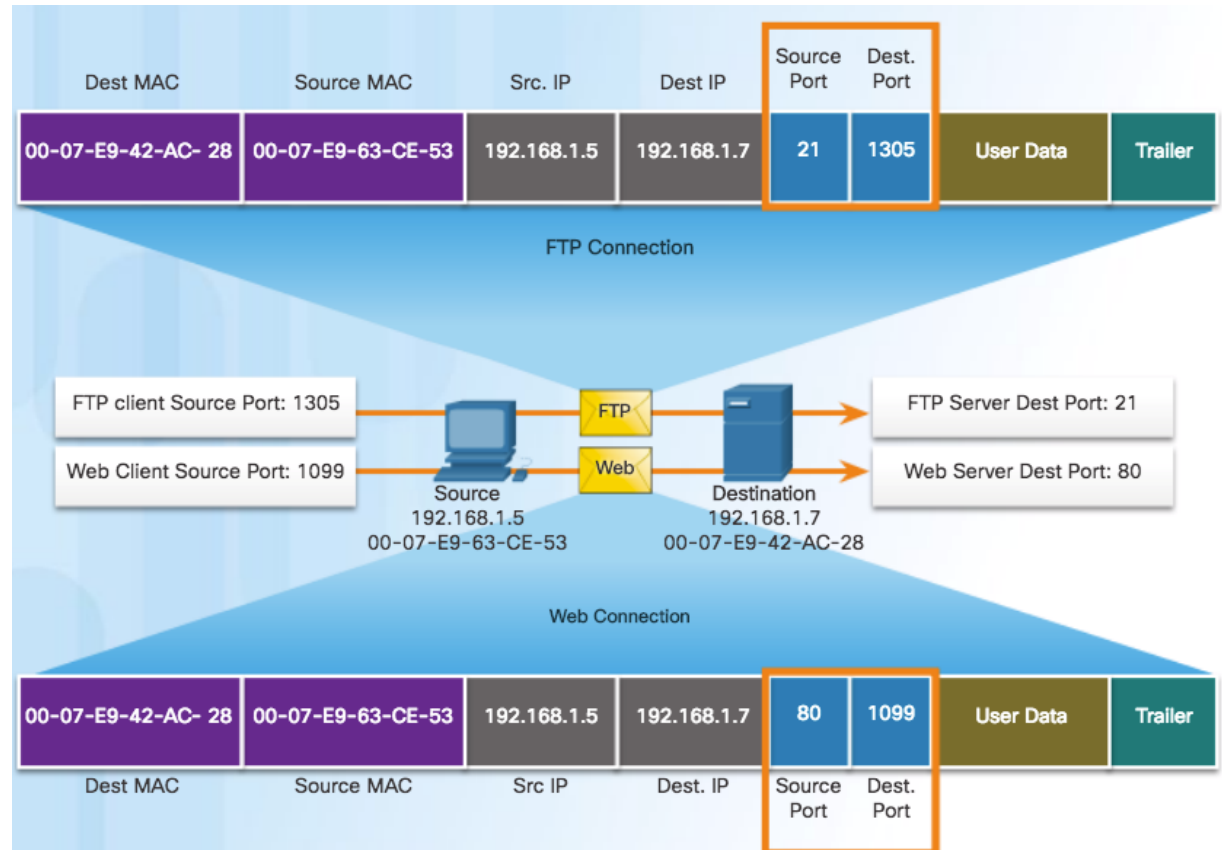
- Tell the destination what service is being requested
- Example: Port 80 web services are being requested



TCP and UDP Overview

Socket Pairs

- Source and destination port placed in segment
- Segments encapsulated in IP packet
- IP and port number = socket
- Example: 192.168.1.7:80
- Sockets enable multiple processes to be distinguished
- Source port acts as a return address



Port Number Groups

Port Number Range	Port Group
0 to 1023	Well-known Ports
1024 to 49151	Registered Ports
49152 to 65535	Private and/or Dynamic Ports

- Well-known Ports (Numbers 0 to 1023) - These numbers are reserved for services and applications.
- Registered Ports (Numbers 1024 to 49151) - These port numbers are assigned by IANA to a requesting entity to use with specific processes or applications.
- Dynamic or Private Ports (Numbers 49152 to 65535) - Usually assigned dynamically by the client's OS and used to identify the client application during communication.

Port Number Groups (Cont.)

Well
Known
Port
Numbers

Port Number	Protocol	Application	Acronym
20	TCP	File Transfer Protocol (data)	FTP
21	TCP	File Transfer Protocol (control)	FTP
22	TCP	Secure Shell	SSH
23	TCP	Telnet	–
25	TCP	Simple Mail Transfer Protocol	SMTP
53	UDP, TCP	Domain Name Service	DNS
67	UDP	Dynamic Host Configuration Protocol (server)	DHCP
68	UDP	Dynamic Host Configuration Protocol (client)	DHCP
69	UDP	Trivial File Transfer Protocol	TFTP
80	TCP	Hypertext Transfer Protocol	HTTP
110	TCP	Post Office Protocol version 3	POP3
143	TCP	Internet Message Access Protocol	IMAP
161	UDP	Simple Network Management Protocol	SNMP
443	TCP	Hypertext Transfer Protocol Secure	HTTPS

The netstat Command

- Network utility that can be used to verify connections
- By default, will attempt to resolve IP addresses to domain names and port numbers to well-known applications
- -n option used to display IPs and ports in numerical form

```
C:\> netstat

Active Connections

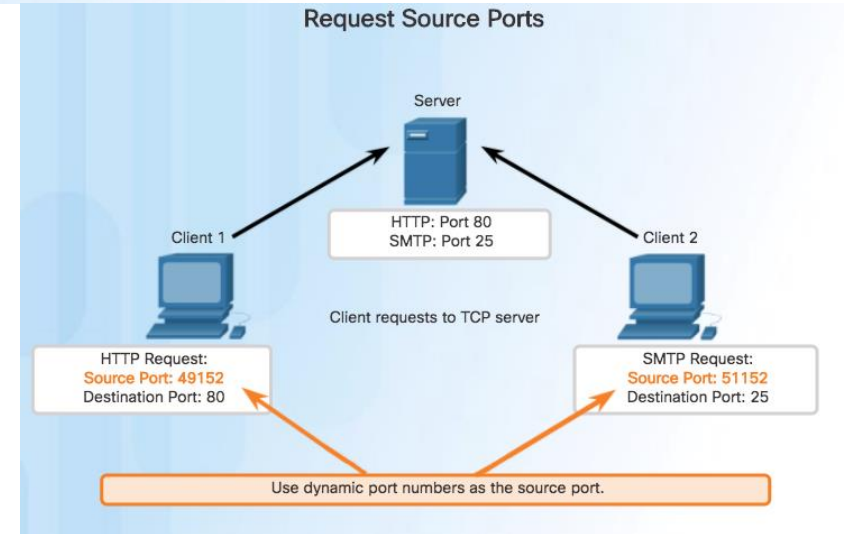
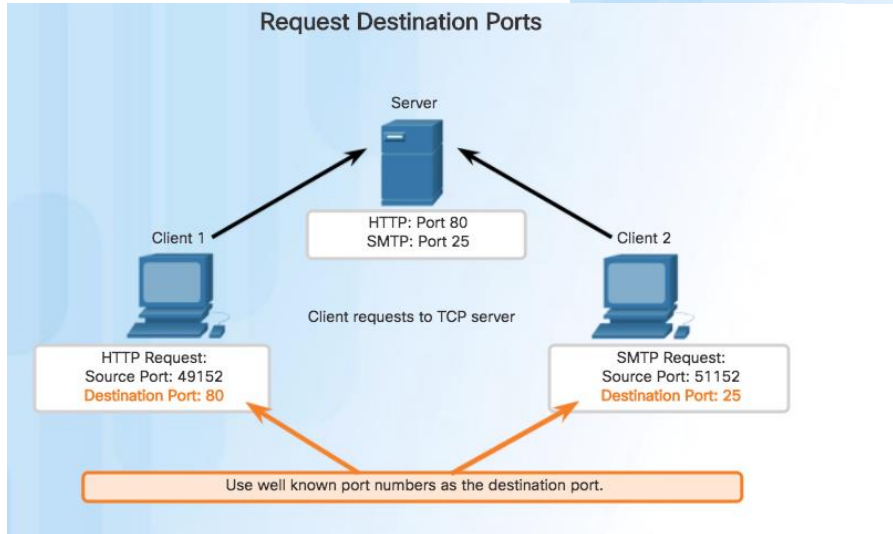
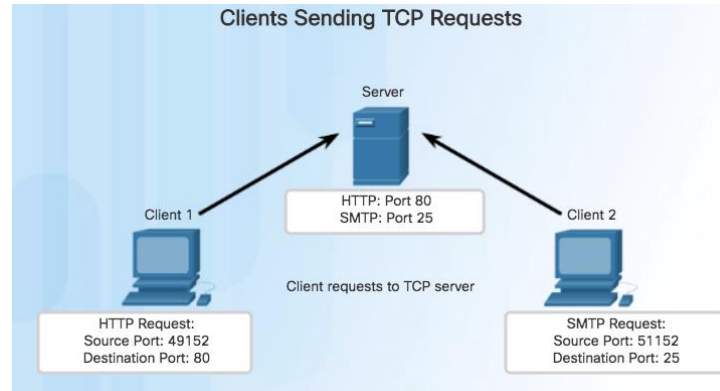
Proto Local Address Foreign Address State
TCP kenpc:3126 192.168.0.2:netbios-ssn ESTABLISHED
TCP kenpc:3158 207.138.126.152:http ESTABLISHED
TCP kenpc:3159 207.138.126.169:http ESTABLISHED
TCP kenpc:3160 207.138.126.169:http ESTABLISHED
TCP kenpc:3161 sc.msn.com:http ESTABLISHED
TCP kenpc:3166 www.cisco.com:http ESTABLISHED

C:\>
```

9.2 TCP and UDP

TCP Communication Process

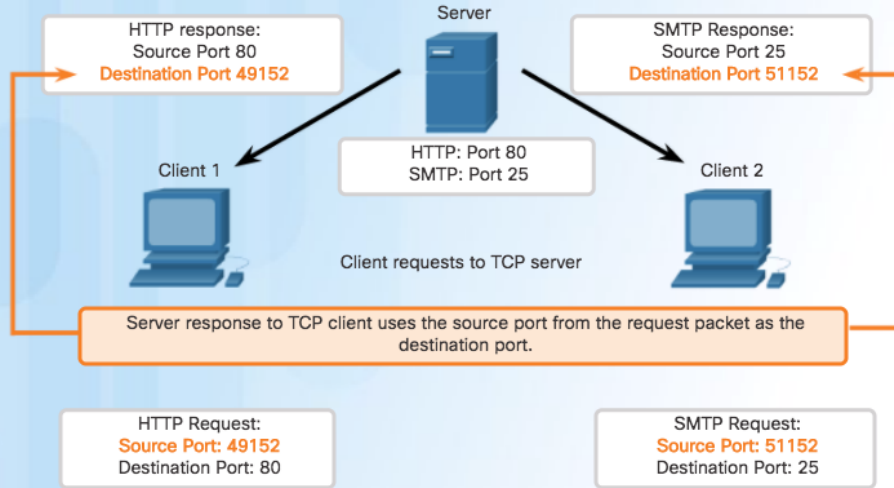
TCP Server Process



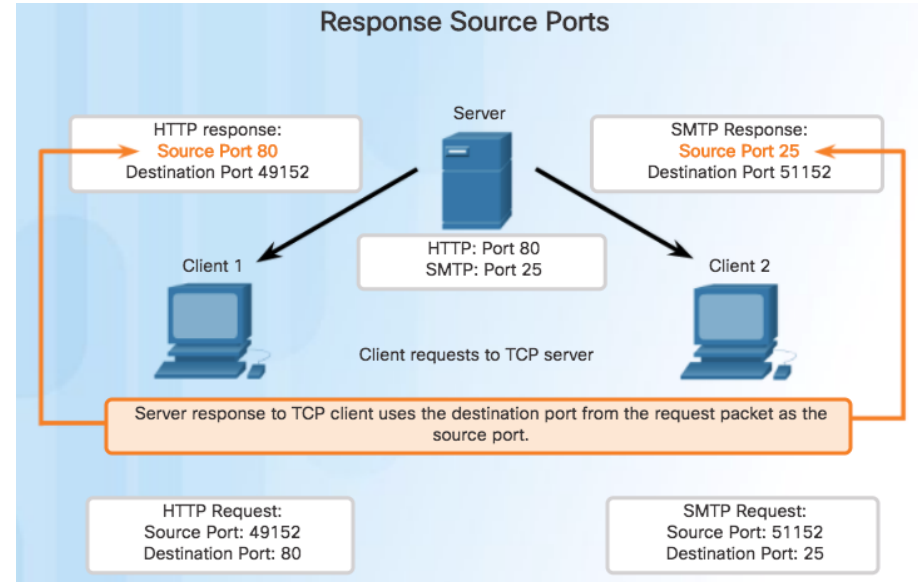
TCP Communication Process

TCP Server Process (Cont.)

Response Destination Ports

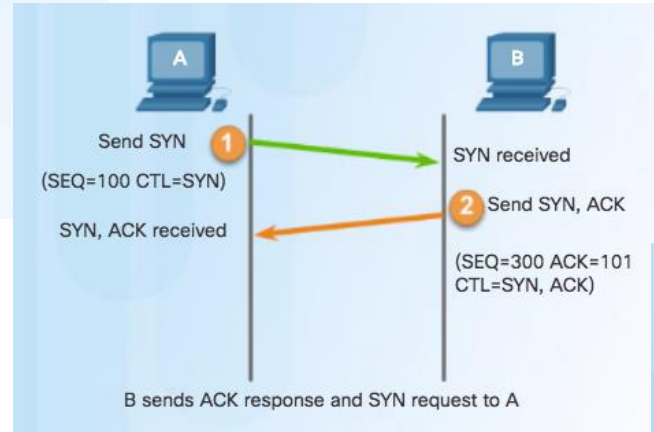
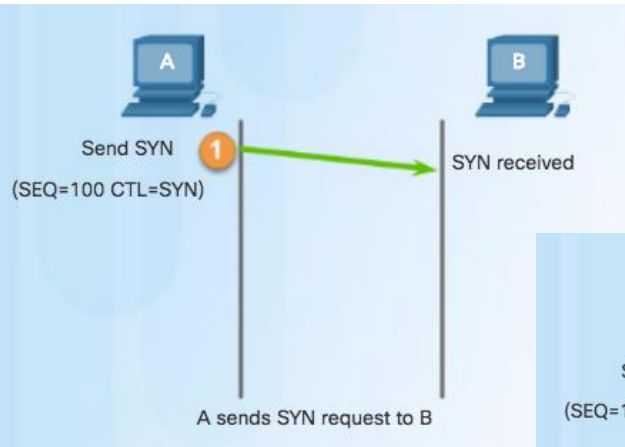


Response Source Ports

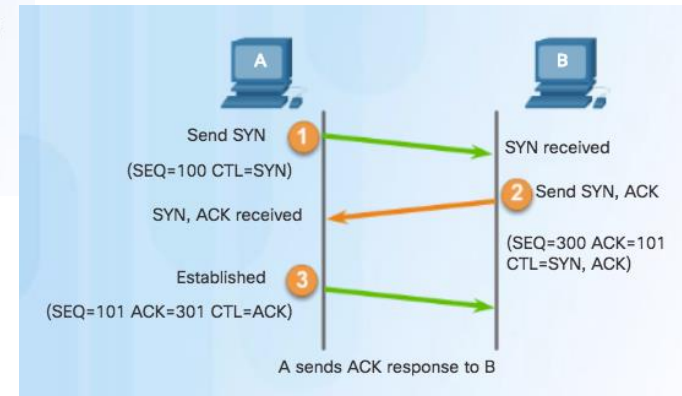


TCP Communication Process

TCP Connection Establishment



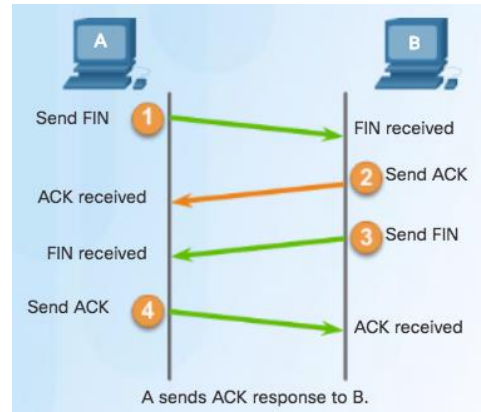
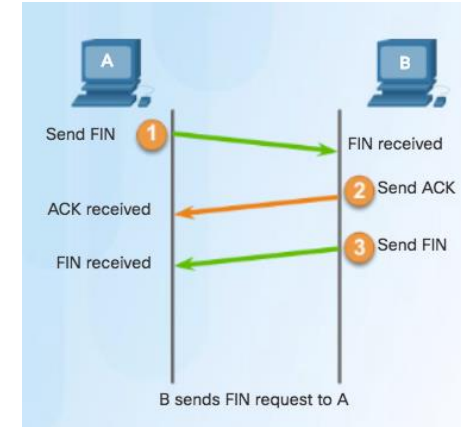
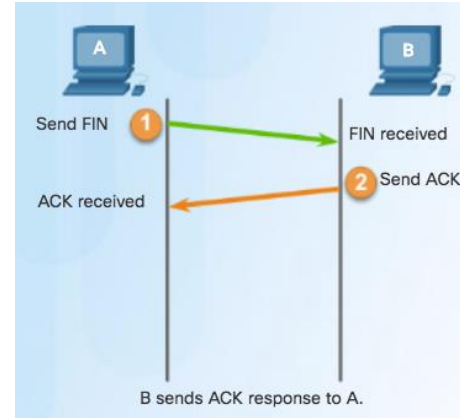
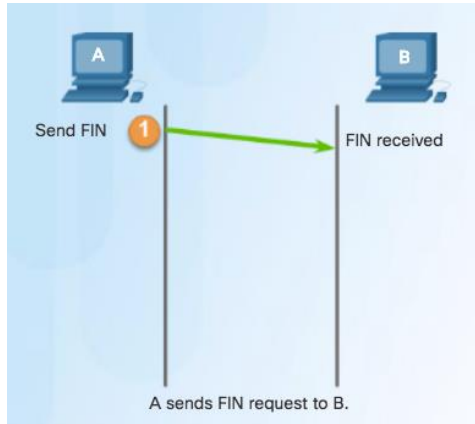
- Step 3 – Client acknowledges communication session with server.



- Step 2 – Server acknowledges and requests a session with client.

TCP Communication Process

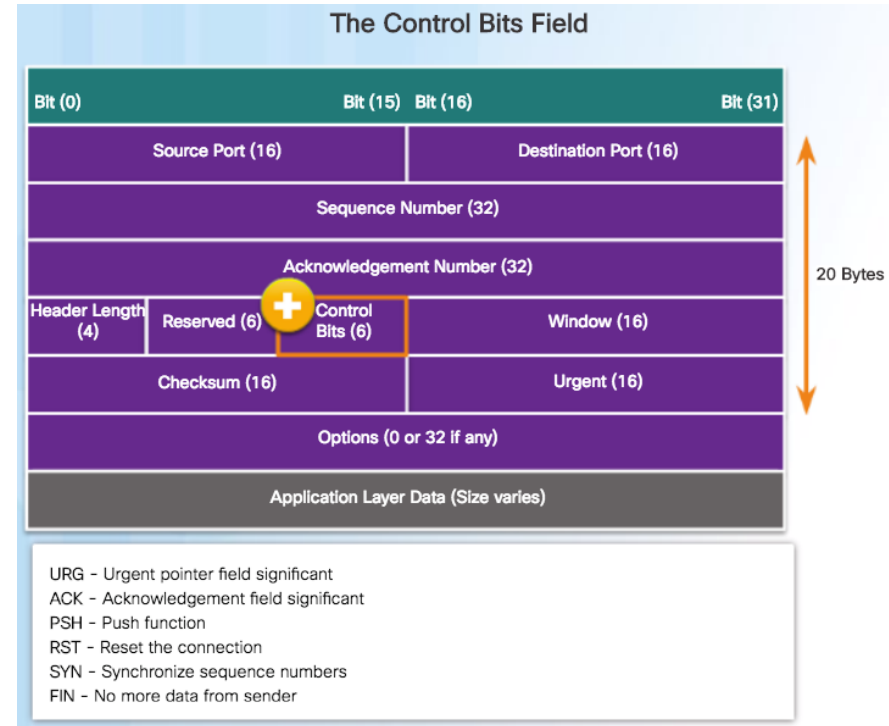
TCP Session Termination



- To close a connection, the Finish (FIN) control flag must be set in the segment header.
- To end each one-way TCP session, a two-way handshake, consisting of a FIN segment and an Acknowledgment (ACK) segment, is used.
- To terminate a single conversation supported by TCP, four exchanges are needed to end both sessions.

TCP Three-way Handshake Analysis

- The three-way handshake:
 - Establishes that the destination device is present on the network.
 - Verifies that the destination device has an active service and is accepting requests on the destination port number that the initiating client intends to use.
 - Informs the destination device that the source client intends to establish a communication session on that port number.
- The six bits in the Control Bits field of the TCP segment header are also known as flags.
 - RST flag is used to reset a connection when an error or timeout occurs



Video Demonstration - TCP 3-Way Handshake

No.	Time	Source	Destination	Protocol	Info
10	16.303490	10.1.1.1	192.168.254.254	TCP	kiosk > http [SYN] Seq=0 W
11	16.304896	192.168.254.254	10.1.1.1	TCP	http > kiosk [SYN, ACK] Seq
12	16.304925	10.1.1.1	192.168.254.254	TCP	kiosk > http [ACK] Seq=1 A
13	16.305153	10.1.1.1	192.168.254.254	HTTP	GET / HTTP/1.1
14	16.307875	192.168.254.254	10.1.1.1	TCP	http > kiosk [ACK] Seq=1 A

■	Frame 10: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
■	Ethernet II, Src: VMware_be:62:88 (00:50:56:be:62:88), Dst: Cisco_63:74:a0 (00:0f:24:63:
■	Internet Protocol Version 4, Src: 10.1.1.1 (10.1.1.1), Dst: 192.168.254.254 (192.168.254:
■	Transmission Control Protocol, Src Port: kiosk (1061), Dst Port: http (80), Seq: 0, Len:
	Source port: kiosk (1061)
	Destination port: http (80)
	[Stream index: 0]
	Sequence number: 0 (relative sequence number)
	Header length: 28 bytes
■	Flags: 0x02 (SYN)
	000. = Reserved: Not set
	...0 = Nonce: Not set
	 0... = Congestion window Reduced (CWR): Not set
	0.. = ECN-Echo: Not set
	0. = Urgent: Not set
	0 = Acknowledgement: Not set
	 0... = Push: Not set
	0.. = Reset: Not set
■	1. = Syn: Set
	0 = Fin: Not set

SYN
SYN, ACK
ACK



Lab – Using Wireshark to Observe the TCP 3-Way Handshake

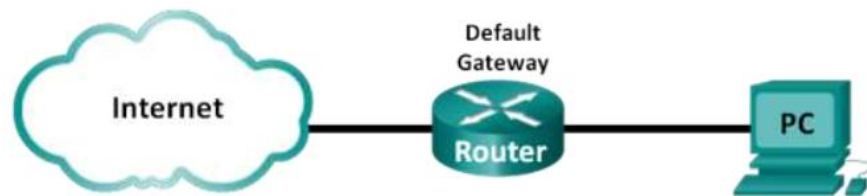


Cisco Networking Academy™

Mind Wide Open™

Lab - Using Wireshark to Observe the TCP 3-Way Handshake

Topology



Objectives

Part 1: Prepare Wireshark to Capture Packets

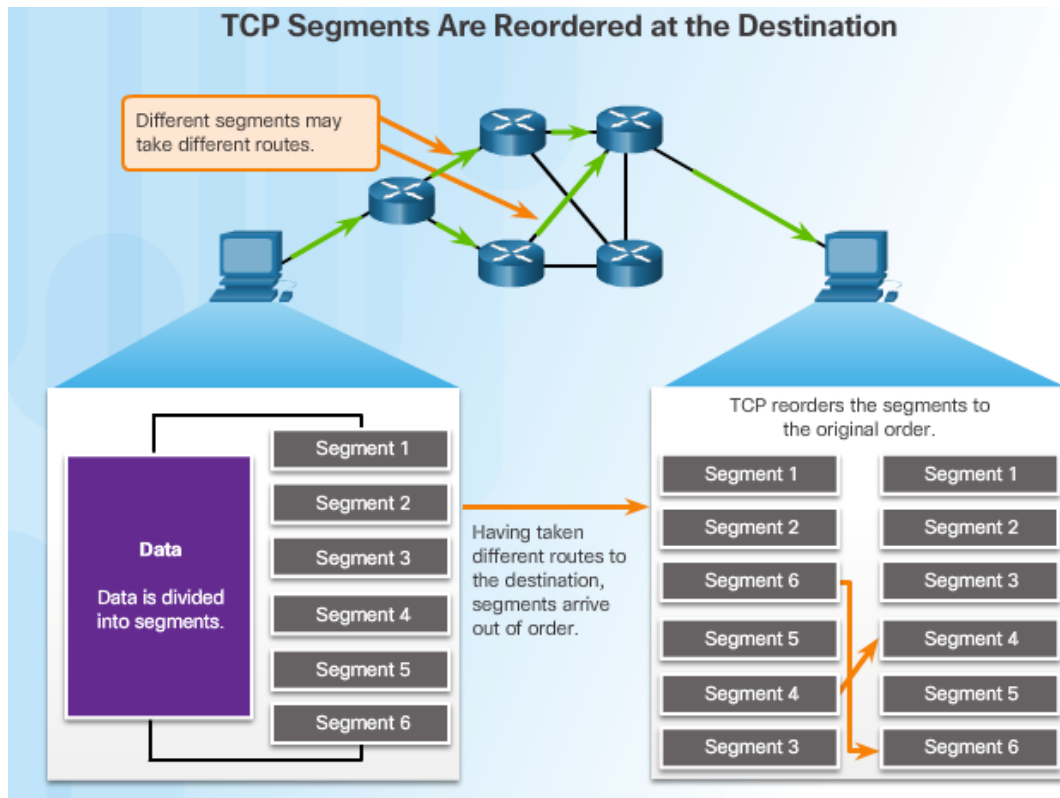
Part 2: Capture, Locate, and Examine Packets

Background / Scenario

In this lab, you will use Wireshark to capture and examine packets generated between the PC browser using the HyperText Transfer Protocol (HTTP) and a web server, such as www.google.com. When an application, such as HTTP or File Transfer Protocol (FTP) first starts on a host, TCP uses the three-way handshake to establish a reliable TCP session between the two hosts. For example, when a PC uses a web browser to surf the Internet, a three-way handshake is initiated, and a session is established between the PC host and web server. A PC can have multiple simultaneous active TCP sessions with various web sites.

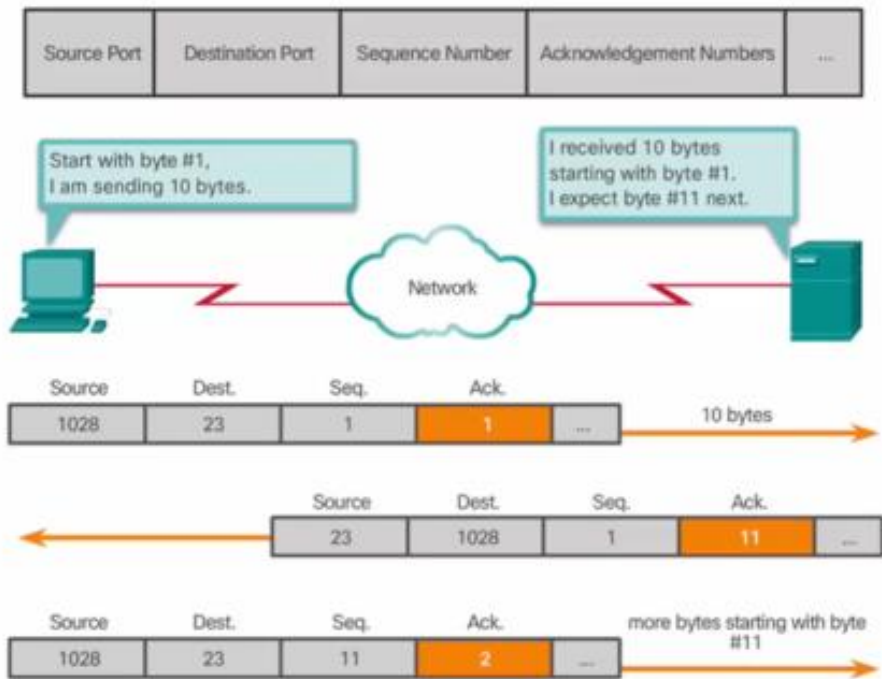
TCP Reliability – Ordered Delivery

- Sequence numbers are assigned in the header of each packet.
- Represents the first data byte of the TCP segment.
- During session setup, an initial sequence number (ISN) is set - represents the starting value of the bytes.
- As data is transmitted during the session, the sequence number is incremented by the number of bytes that have been transmitted.
- Missing segments can then be identified.



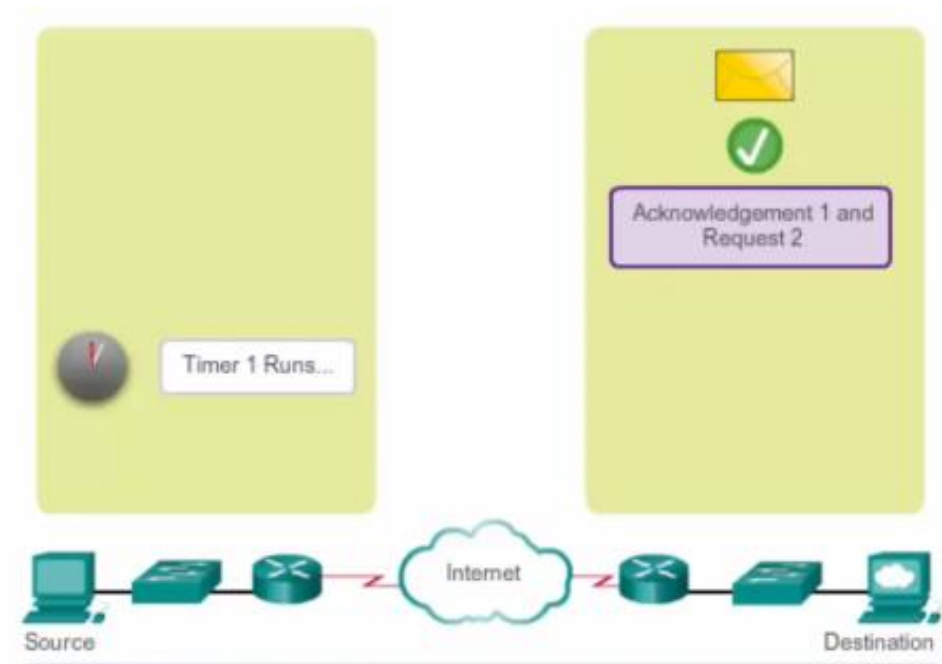
Reliability and Flow Control

Video Demonstration - TCP Reliability – Sequence Numbers and Acknowledgments



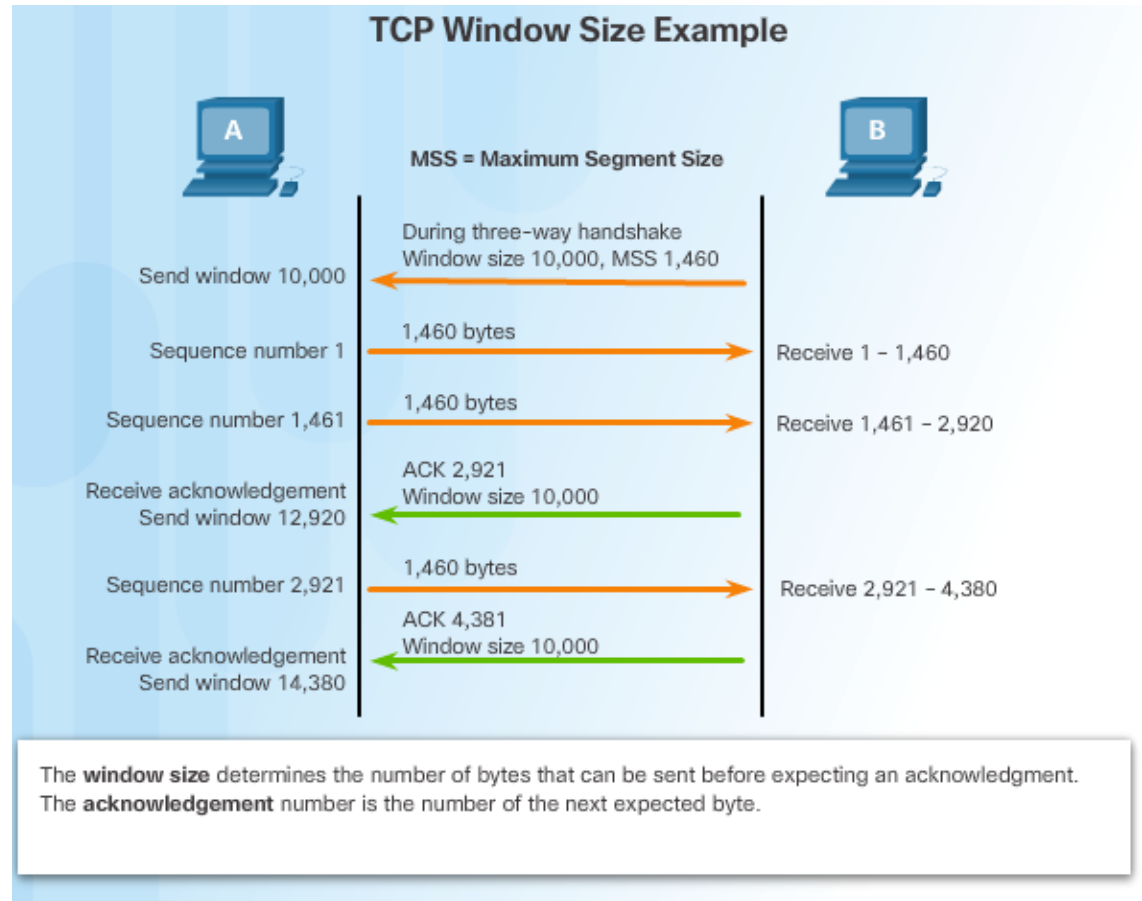
Reliability and Flow Control

Video Demonstration – Data Loss and Retransmission



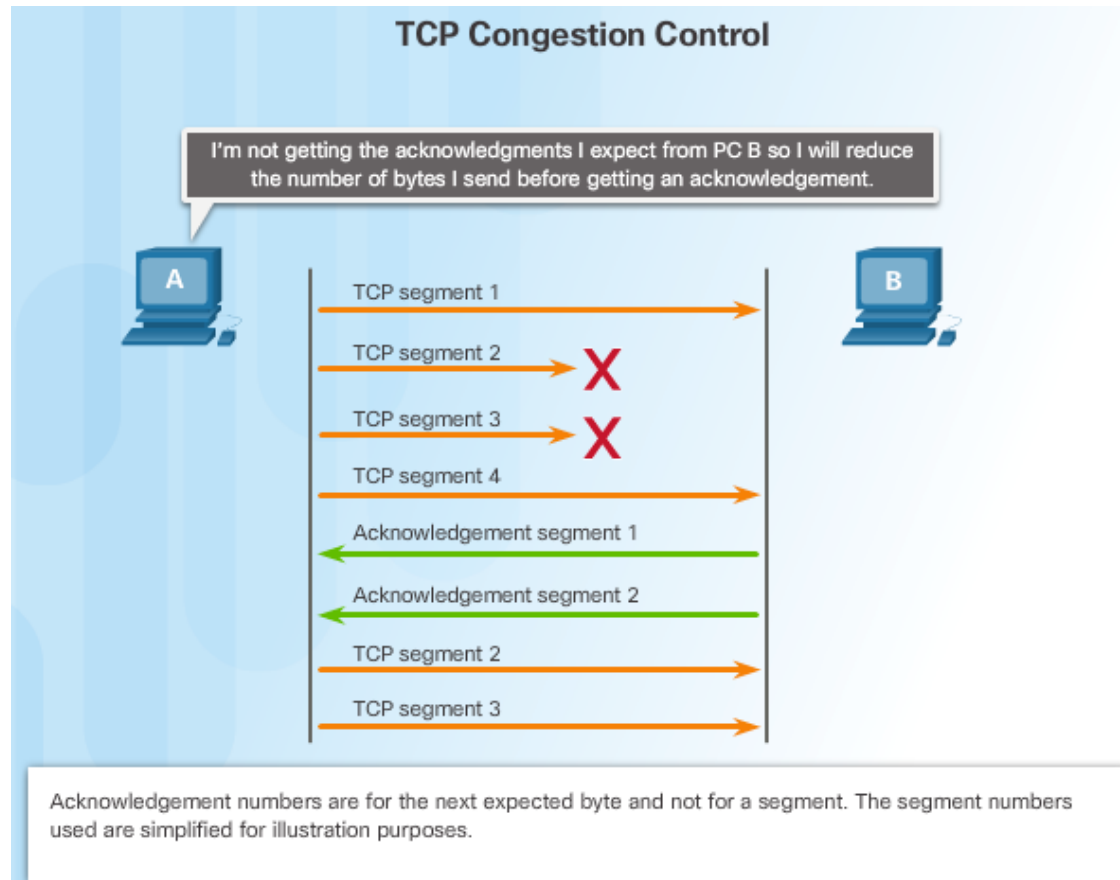
TCP Flow Control – Window Size and Acknowledgments

- In the figure, the source is transmitting 1,460 bytes of data within each segment.
- Window size agreed on during 3-way handshake.
- Typically, PC B will not wait for 10,000 bytes before sending an acknowledgment.
- PC A can adjust its send window as it receives acknowledgments from PC B.



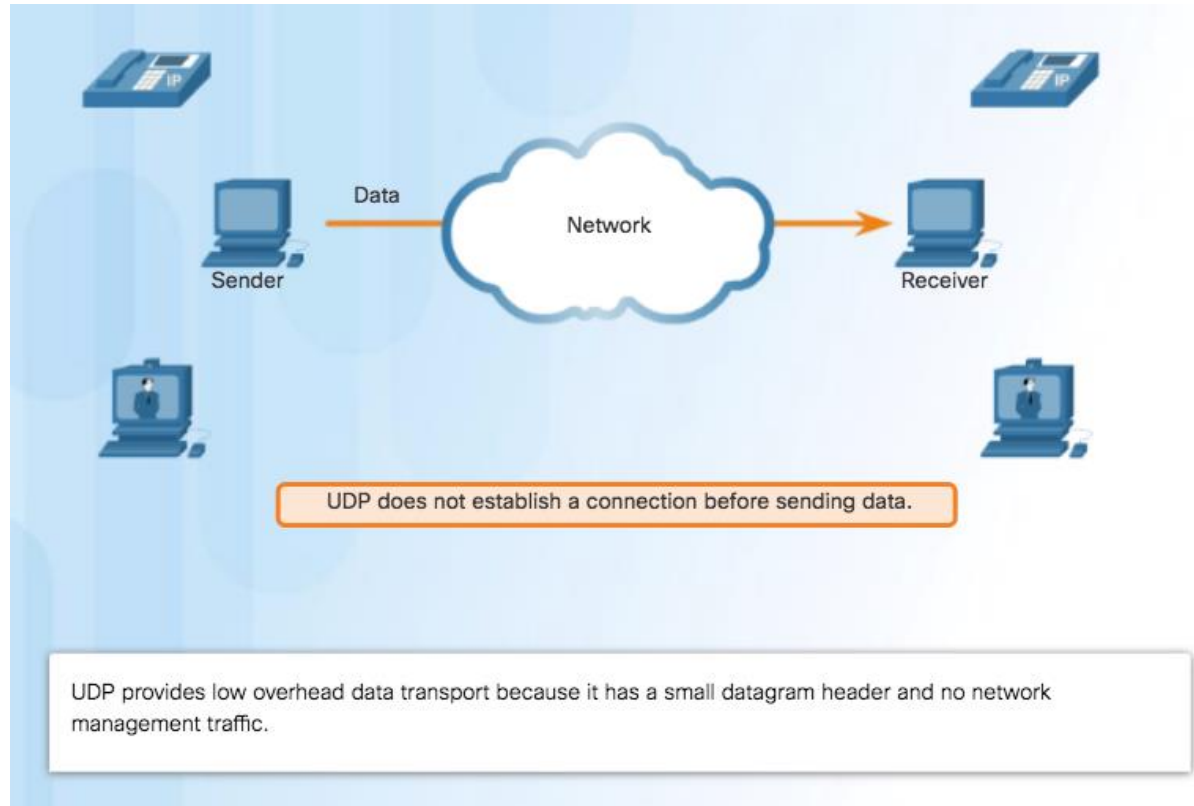
TCP Flow Control – Congestion Avoidance

- Congestion causes retransmission of lost TCP segments
- Retransmission of segments can make the congestion worse
- To avoid and control congestion, TCP employs several congestion handling mechanisms, timers, and algorithms
- Example: Reduce the number of bytes it sends before receiving an acknowledgment



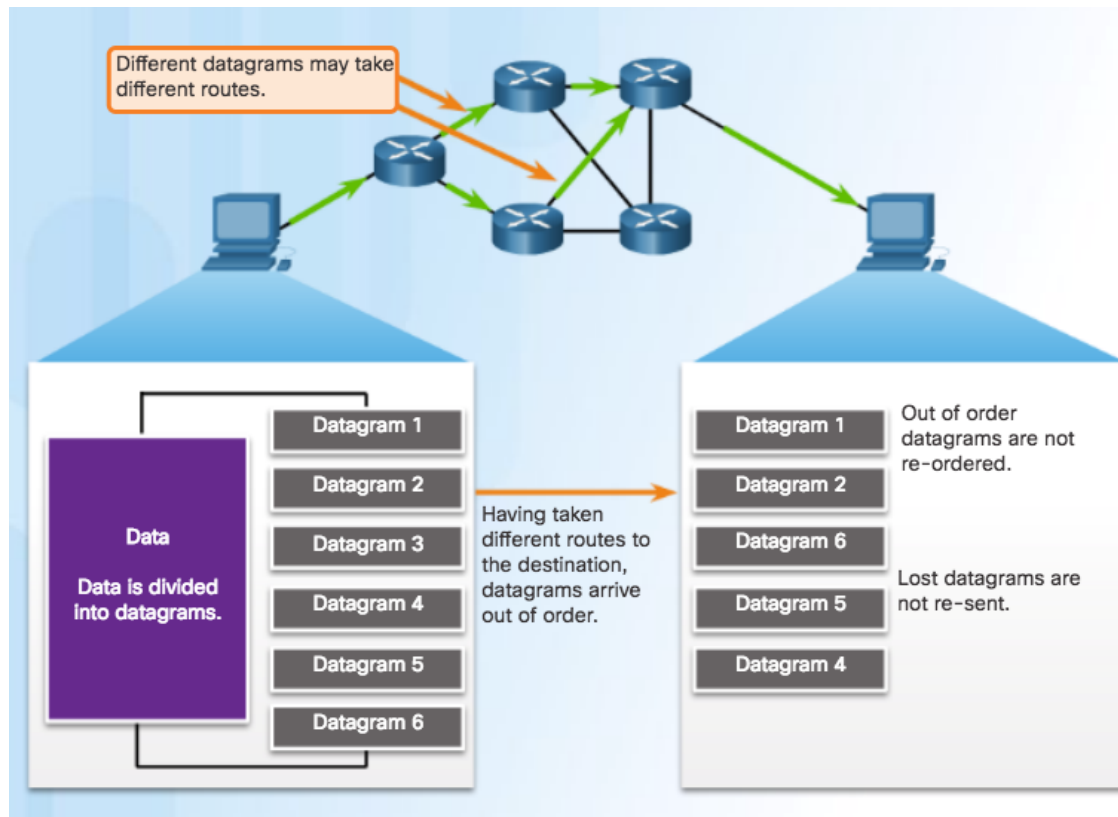
UDP Low Overhead versus Reliability

- UDP not connection-oriented
- No retransmission, sequencing, and flow control
- Functions not provided by the transport layer implemented elsewhere



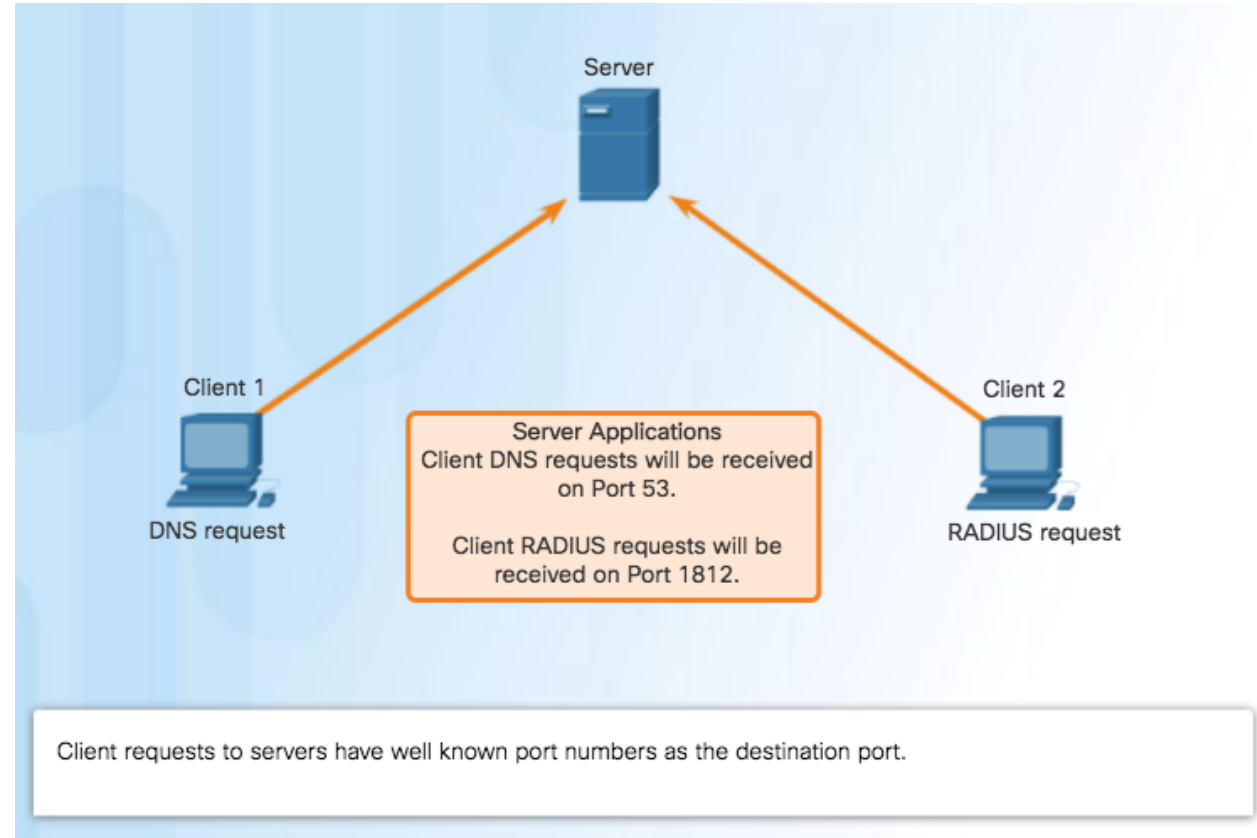
UDP Datagram Reassembly

- UDP reassembles data in order received and forwards to application
- Application must identify the proper sequence



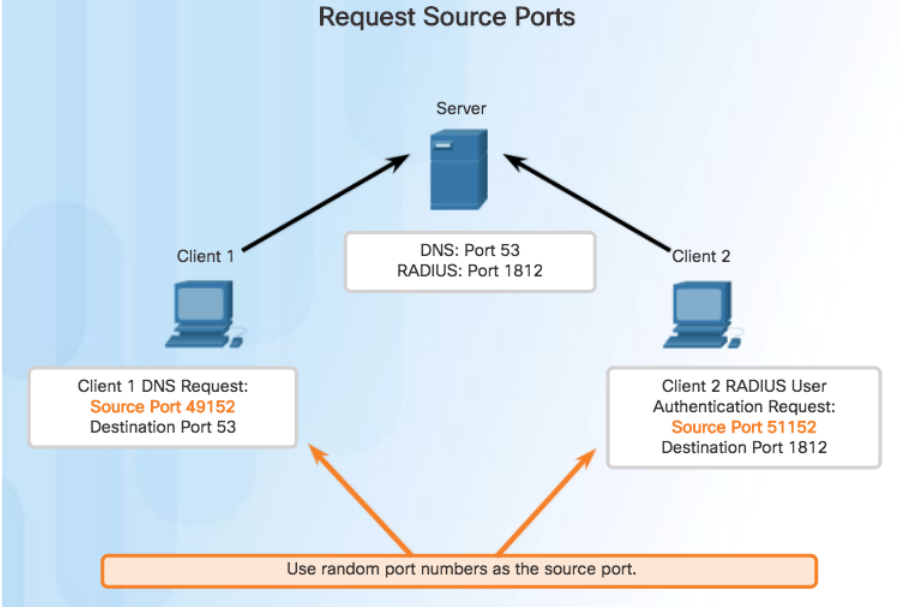
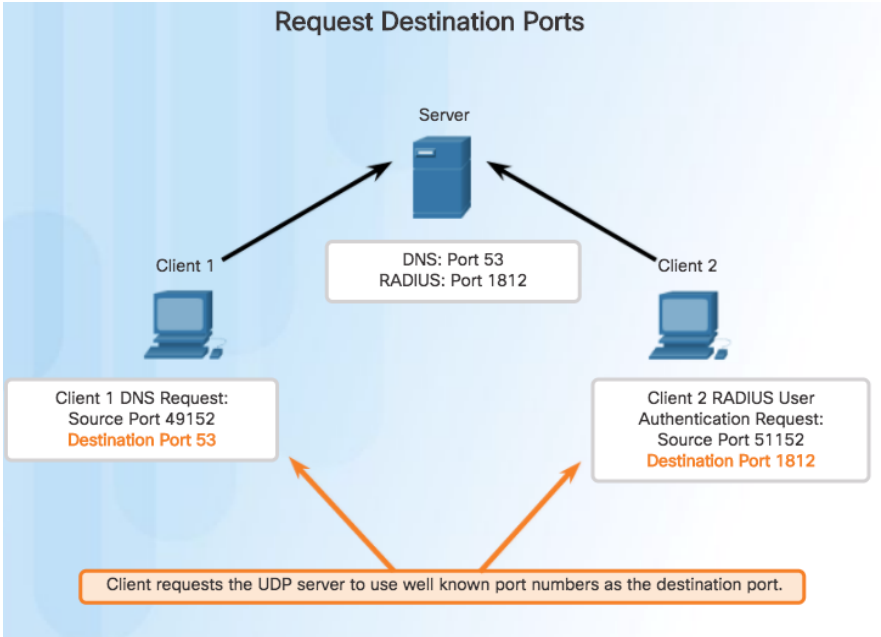
UDP Server Processes and Requests

Note: The Remote Authentication Dial-in User Service (RADIUS) server shown in the figure provides authentication, authorization, and accounting services to manage user access.



UDP Communication

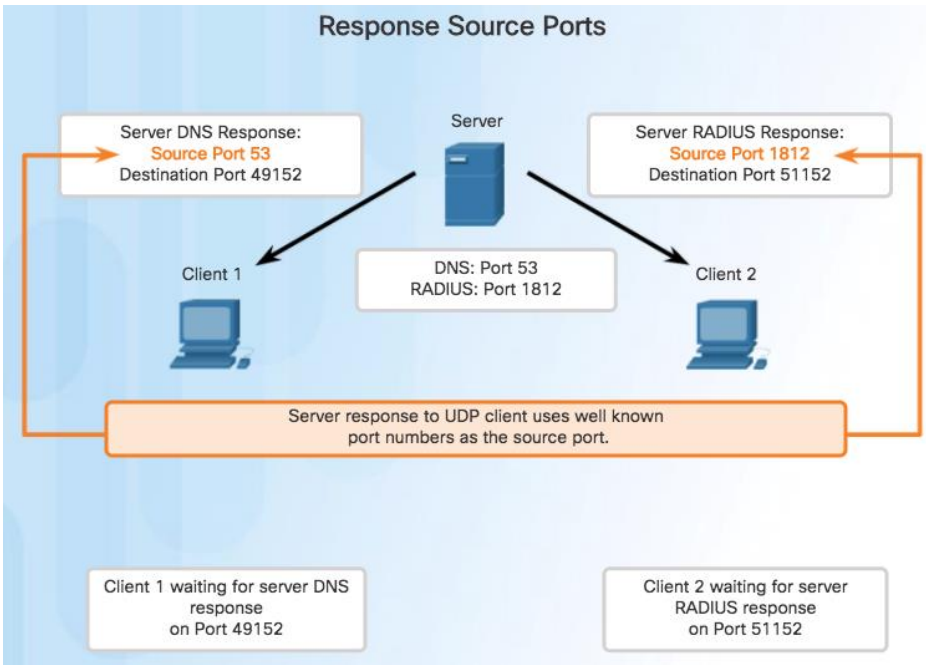
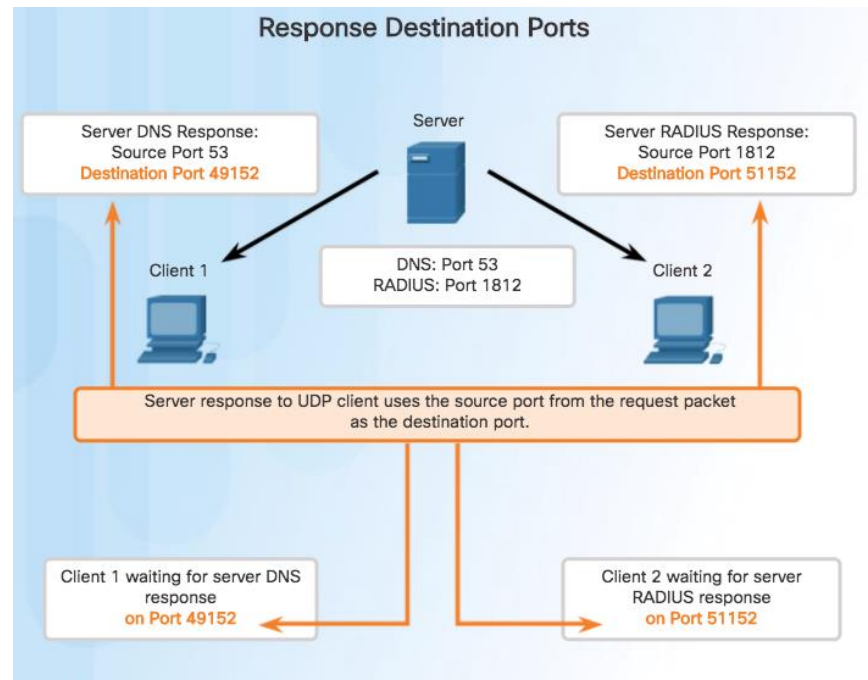
UDP Client Processes



Clients Sending UDP Requests

UDP Communication

UDP Client Processes (Cont.)



Clients Sending UDP Requests

Lab – Using Wireshark to Examine a UDP DNS Capture

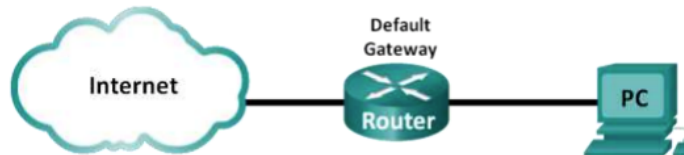


Cisco Networking Academy®

Mind Wide Open™

Lab - Using Wireshark to Examine a UDP DNS Capture

Topology



Objectives

- Part 1: Record a PC's IP Configuration Information
- Part 2: Use Wireshark to Capture DNS Queries and Responses
- Part 3: Analyze Captured DNS or UDP Packets

Background / Scenario

If you have ever used the Internet, you have used the Domain Name System (DNS). DNS is a distributed network of servers that translates user-friendly domain names like `www.google.com` to an IP address. When you type a website URL into your browser, your PC performs a DNS query to the DNS server's IP address. Your PC's DNS server query and the DNS server's response make use of the User Datagram Protocol (UDP) as the transport layer protocol. UDP is connectionless and does not require a session setup as does TCP. DNS queries and responses are very small and do not require the overhead of TCP.

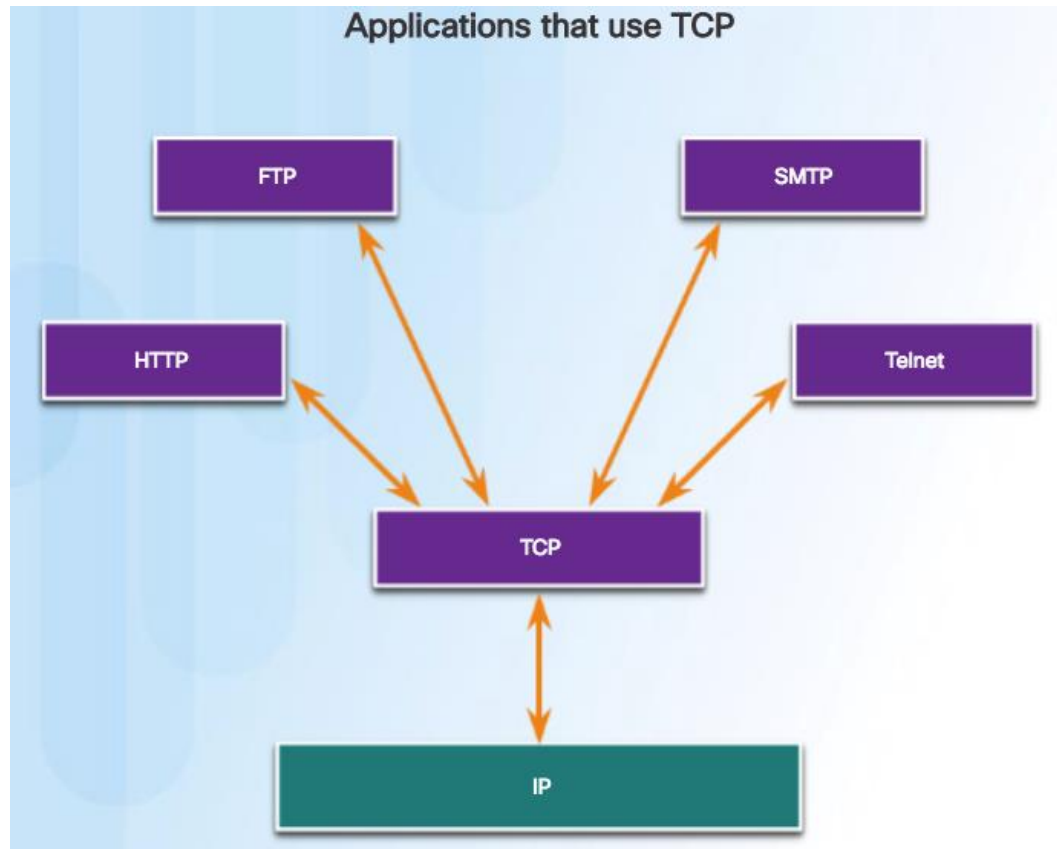
In this lab, you will communicate with a DNS server by sending a DNS query using the UDP transport protocol. You will use Wireshark to examine the DNS query and response exchanges with the same server.

Note: This lab cannot be completed using Netlab. This lab assumes that you have Internet access.

Required Resources

Applications that use TCP

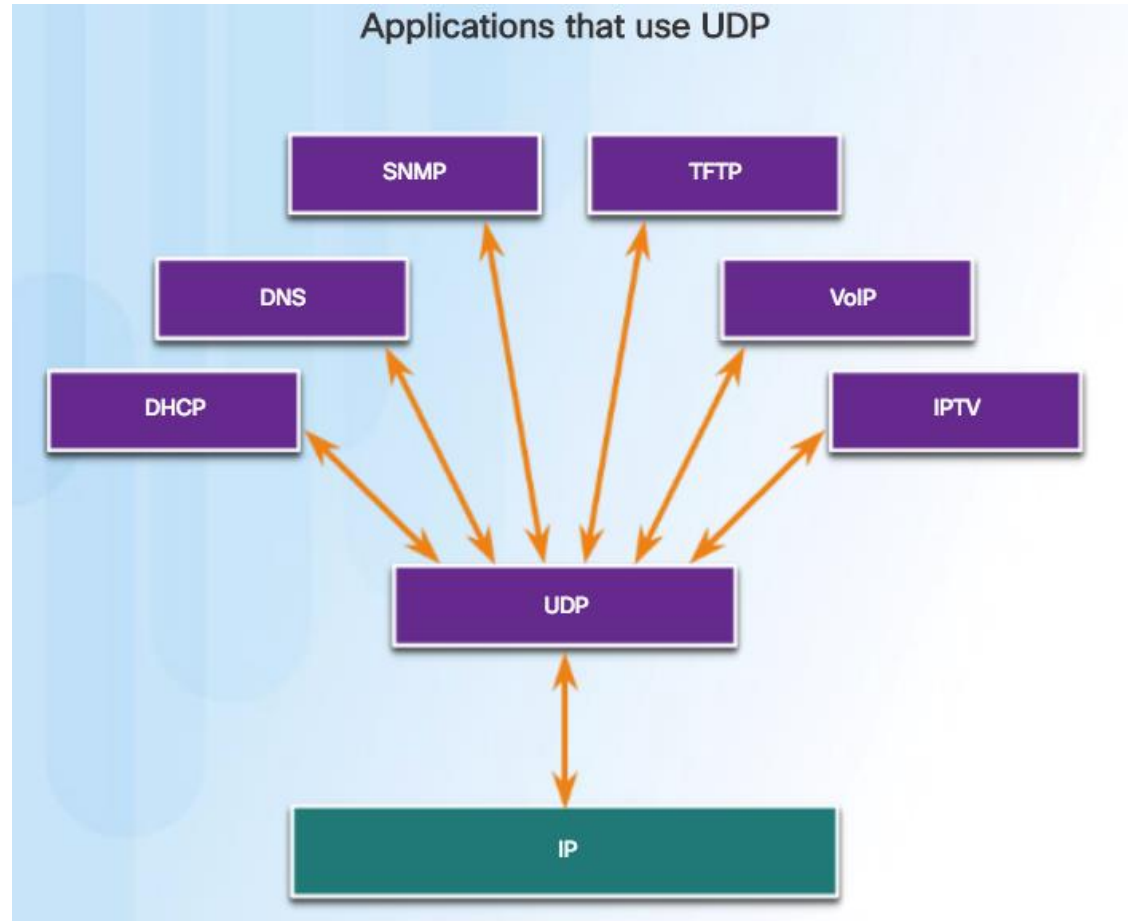
TCP frees applications from having to manage reliability



Applications that use UDP

Three types of applications best suited for UDP:

- Live video and multimedia
- Simple request and reply
- Handle reliability themselves



Lab – Using Wireshark to Examine TCP and UDP Captures



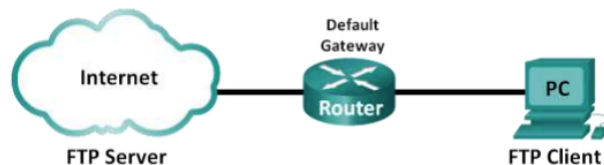
Cisco Networking Academy®

Mind Wide Open™

Lab - Using Wireshark to Examine TCP and UDP Captures

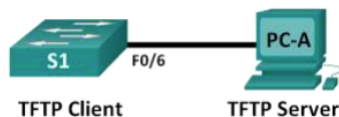
Topology – Part 1 (FTP)

Part 1 will highlight a TCP capture of an FTP session. This topology consists of a PC with Internet access.



Topology – Part 2 (TFTP)

Part 2 will highlight a UDP capture of a TFTP session. The PC must have both an Ethernet connection and a console connection to Switch S1.



Addressing Table (Part 2)

Device	Interface	IP Address	Subnet Mask	Default Gateway
S1	VLAN 1	192.168.1.1	255.255.255.0	N/A
PC-A	NIC	192.168.1.3	255.255.255.0	192.168.1.1

Objectives

Part 1: Identify TCP Header Fields and Operation Using a Wireshark FTP Session Capture

Part 2: Identify UDP Header Fields and Operation Using a Wireshark TFTP Session Capture

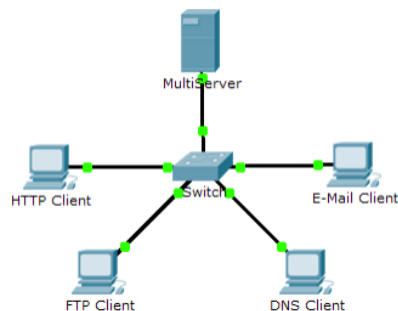
9.3 Chapter Summary

Packet Tracer – TCP and UDP Communications



Packet Tracer Simulation - TCP and UDP Communications

Topology



Objectives

Part 1: Generate Network Traffic in Simulation Mode

Part 2: Examine the Functionality of the TCP and UDP Protocols

Background

This simulation activity is intended to provide a foundation for understanding the TCP and UDP in detail. Simulation mode provides the ability to view the functionality of the different protocols.

As data moves through the network, it is broken down into smaller pieces and identified in some fashion so that the pieces can be put back together. Each of these pieces is assigned a specific name (protocol data unit [PDU]) and associated with a specific layer. Packet Tracer Simulation mode enables the user to view each of the protocols and the associated PDU. The steps outlined below lead the user through the process of requesting services using various applications available on a client PC.

This activity provides an opportunity to explore the functionality of the TCP and UDP protocols, multiplexing and the function of port numbers in determining which local application requested the data or is sending the data.

Chapter 9: Transport Layer

- Explain how transport layer protocols and services support communications across data networks.
- Compare the operations of transport layer protocols in supporting end-to-end communication.

